Programming Fundamentals

Looping

Arab Academy
for Science, Technology and Maritime Transport

Lecture Outline:

- The increment and decrement Operators
- The While loops
- The for loops
- The do while loops

The increment and decrement Operators

- To increment a value means to increase it by one, and to decrement a value means to decrease it by one.
- ++ and -- are operators that add and subtract 1 from their operands

```
num = num + 1;
num += 1;
```

→ **num++;**

++num;

```
num = num - 1;
num -= 1;
```

→ **num--;**

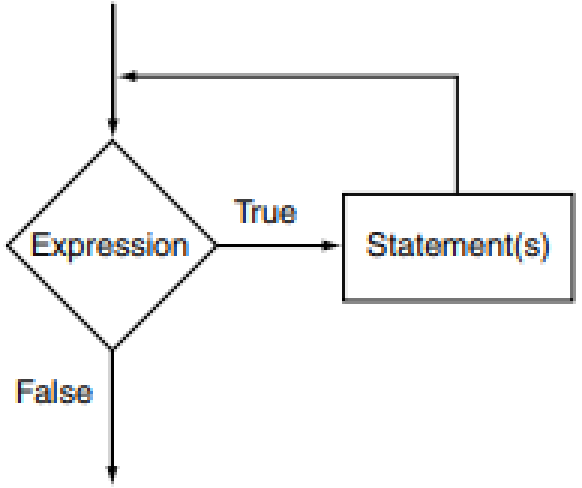
--num;

The While loops

- The while loop has two important parts:

(1) an expression that is tested for a true or false value,

(2) a statement or block that is repeated as long as the expression is true.



The While loops

- Here is the general format of the while loop:

```
while (expression)
  statement;
```

- Here's how the loop works: the expression is tested, and if it is true, the statement is executed. Then, the expression is tested again. If it is true, the statement is executed. This cycle repeats until the expression is false.
- Notice there is no semicolon after the expression in parentheses

The While loops

- If you wish the while loop to repeat a block of statements, its format is

```
while (expression)
{
    statement;
    statement;
    // Place as many statements here
    // as necessary.
}
```

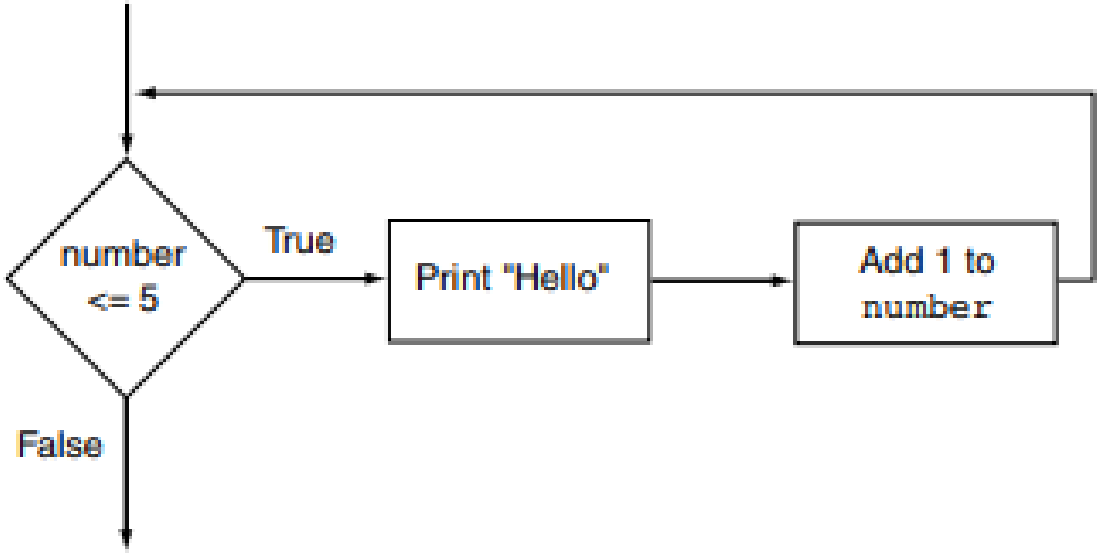
The While loops

Test this expression.

If the expression is true, perform these statements.

```
while (number <= 5)
{
    cout << "Hello\n";
    number++;
}
```

After executing the body of the loop, start over.



The While loops

Program 5-3

```
// This program demonstrates a simple while loop.
#include <iostream>
using namespace std;

int main()
{
    int number = 1;

    while (number <= 5)
    {
        cout << "Hello\n";
        number++;
    }
    cout << "That's all!\n";
    return 0;
}
```

Program Output

```
Hello
Hello
Hello
Hello
Hello
That's all!
```

The While loops

- The while loop consists of 3 major parts:
  - Initial value
  - Stopping condition
  - update

Initial value

Stopping condition

```
int main()
{
    int number = 1;

    while (number <= 5)
    {
        cout << "Hello\n";
        number++;
    }
    cout << "That's all!\n";
    return 0;
}
```

update

The While loops

- Cases that causes infinite loops

```
int number = 1;
while (number <= 5);  // This semicolon is an ERROR!
{
   cout << "Hello\n";
   number++;
}
```

```
int number = 1;
while (number <= 5)
{
   cout << "Hello\n";
}
```

```
int number = 1;
// This loop is missing its braces!
while (number <= 5)
   cout << "Hello\n";
   number++;
```

The While loops

Example: prints numbers from 1 to 5

```
// C++ Program to print numbers from 1 to 5

#include <iostream>

using namespace std;

int main() {
    int i = 1;

    // while loop from 1 to 5
    while (i <= 5) {
        cout << i << " ";
        ++i;
    }

    return 0;
}
```

The While loops

Example: prints the factorial of n

```
#include<iostream.h>

void main()
{
    int n,f=1,i=1;


    cout<<"\n Enter The Number:";
    cin>>n;

    //LOOP TO CALCULATE THE FACTORIAL OF A NUMBER
    while(i<=n)
    {
        f=f*i;
        i++;
    }

    cout<<"\n The Factorial of "<<n<<" is "<<f;

}
```

The While loops

lets solve this together:

Sum of Numbers

Write a program that asks the user for a positive integer value. The program should use a loop to get the sum of all the integers from 1 up to the number entered. For example, if the user enters 50, the loop will find the sum of 1, 2, 3, 4, ... 50.

Input Validation: Do not accept a negative starting number.

The While loops

lets solve this together:

```
#include<iostream>
using namespace std;
int main() {
int i, num, sum;
cout<<“Enter a positive integer number:”;
cin>>num;
if (num>0) {
        i=1; sum=0;
        While(i<=num)  {
                sum+=i;
                i++;
        }
        cout<<“\n Sum of all positive integer number from 1 to ”<<num<<“is ”<<sum;
}
else
cout<<“\n The number is zero or negative value”;
return 0;
}
```

Exercise #1:
Trace the following programs with the following values 6,12,7,-2,0,-5

```
int main() {
   int number;
   int sum = 0;
   cout << "Enter a number: ";
   cin >> number;
   while (number >= 0)
   {    sum += number;
        cout << "Enter a number: ";
        cin >> number;
   }
   cout << "\nThe sum is " << sum << endl;
     return 0;
}
```

The While loops

lets solve this together:

```
#include<iostream>
using namespace std;
int main() {
int i, num, sum;
cout<<“Enter a positive integer number:”;
cin>>num;
if (num>0) {
        i=1; sum=0;
        While(i<=num)  {
                sum+=i;
                i++;
        }
        cout<<“\n Sum of all positive integer number from 1 to ”<<num<<“is ”<<sum;
}
else
cout<<“\n The number is zero or negative value”;
return 0;
}
```

The While loops

lets solve this together:

Write a program that reads N integer values and prints the average of the even values ONLY.

```
#include<iostream>
using namespace std;
int main() {
int i, c, n, x, sum, aver;
cout<<"Enter a positive integer number:"; cin>>n;
i=1; c=0; sum=0;
While(i<=n)  {
        cout<<"enter a number:"; cin>>x;
        if(x%2==0) {
                sum+=x;
                c++;
        }
        i++;
}
aver=sum/c;
cout<<"\n The average of even values is"<<aver;
return 0;
}
```

The for loops

- Here is the format of the for loop when it is used to repeat a single statement:

```
for (initialization; test; update)
   statement;
```

- The format of the for loop when it is used to repeat a block is

```
for (initialization; test; update)
{
   statement;
   statement;
   // Place as many statements here
   // as necessary.
}
```

The for loops

- Converting from while statement to for statement

Initialization expression

Test expression

```
int num = 1;
while (num <= 10)
{
    cout << num << "\t\t" << (num * num) << endl;
    num++;
}
```

Update expression

Initialization expression | Test expression | Update expression

```
for (num = 1; num <= 10; num++)
    cout << num << "\t\t" << (num * num) << endl;
```

The for loops

- Lets try this too

```
// This program demonstrates a simple while loop.
#include <iostream>
using namespace std;

int main()
{
   int number = 1;

   while (number <= 5)
   {
      cout << "Hello\n";
      number++;
   }
   cout << "That's all!\n";
   return 0;
}
```

```
int main()
{
int number;
for(number=1; number<=30; number++)
{
        cout<<“Hello”;
}
cout<<“That’s all!\n”;
return 0;
}
```

The for loops

Program 5-9

```
1  // This program displays the numbers 1 through 10 and
2  // their squares.
3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8     int num;
9
10    cout << "Number     Number Squared\n";
11    cout << "--------------------------\n";
12
13    for (num = 1; num <= 10; num++)
14       cout << num << "\t\t" << (num * num) << endl;
15    return 0;
16 }
```

Program Output

```
Number      Number Squared
--------------------------
1                 1
2                 4
3                 9
4                 16
5                 25
6                 36
7                 49
8                 64
9                 81
10                100
```

The for loops

lets solve this together:

Calories Burned

Running on a particular treadmill you burn 3.9 calories per minute. Write a program that uses a loop to display the number of calories burned after 10, 15, 20, 25, and 30 minutes.

The for loops

lets solve this together:

```
#include<iostream>
using namespace std;
int main()
{
int i;
for(i=10;i<=30;i+=5)
        cout<<“no. of calories burned after”<<i<<“minutes is”<<(i*3.9)<<endl;
return 0;
}
```

The for loops

lets solve this together:

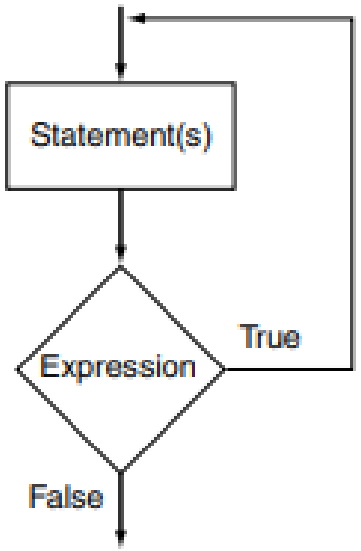
```
#include<iostream>
#include<cmath>
using namespace std;
int main() {
int i, n;
float x, s;
cout<<"Enter your data:";
cin>>x>>n;
s=0;
for(i=1; i<=n ; i++)
    s+=(i / pow(x,i));
cout<<"\nSum="<<s;
return 0;
}
```

Write a c++ program to calculate the sum of n terms of the following serious

$$S=\frac{1}{x^1}+\frac{2}{x^2}+\frac{3}{x^3}+\ldots\ldots\ldots+\frac{n}{x^n}$$

The do-while loops

- The do-while loop is a posttest loop, which means its expression is tested after each iteration.



- Here is the do-while loop's format when the body of the loop contains only a single statement:

```
do
   statement;
while (expression);
```

- The do-while loop must be terminated with a semicolon

The do-while loops

- Here is the format of the do-while loop when the body of the loop contains multiple statements

```
do
{
  statement;
  statement;
  // Place as many statements here
  // as necessary.
} while (expression);
```

The do-while loops

- The do-while loop always performs at least one iteration, even if the expression is false to begin with. This differs from the behavior of a while loop.

```
int x = 1;
while (x < 0)
    cout << x << endl;
```

```
int x = 1;
do
    cout << x << endl;
while (x < 0);
```

The do-while loops

```
// This program averages 3 test scores. It repeats as
// many times as the user wishes.
#include <iostream>
using namespace std;

int main()
{
   int score1, score2, score3; // Three scores
   double average;             // Average score
   char again;                 // To hold Y or N input

   do
   {
      // Get three scores.
      cout << "Enter 3 scores and I will average them: ";
      cin >> score1 >> score2 >> score3;

      // Calculate and display the average.
```

The do-while loops

```
        average = (score1 + score2 + score3) / 3.0;
        cout << "The average is " << average << ".\n";

        // Does the user want to average another set?
        cout << "Do you want to average another set? (Y/N) ";
        cin >> again;
    } while (again == 'Y' || again == 'y');
    return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter 3 scores and I will average them: 80 90 70 [Enter]
The average is 80.
Do you want to average another set? (Y/N) y [Enter]
Enter 3 scores and I will average them: 60 75 88 [Enter]
The average is 74.3333.
Do you want to average another set? (Y/N) n [Enter]
```

Thank You